



Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний
інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE »**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати функціонал для роботи в розподіленому оточенні (логіку роботи).....	3
2. Реалізувати взаємодію розподілених частин.....	6
Висновки:.....	9
Код проекту:.....	9

Тема:

РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Мета:

Ознайомитися з основними принципами та шаблонами взаємодії між додатками, такими як «Client-Server», «Peer-to-Peer» та «Service-Oriented Architecture», дослідити їхні особливості та способи реалізації для створення гнучких, масштабованих і ефективних програмних систем, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати функціонал для роботи в розподіленому оточенні (логіку роботи)

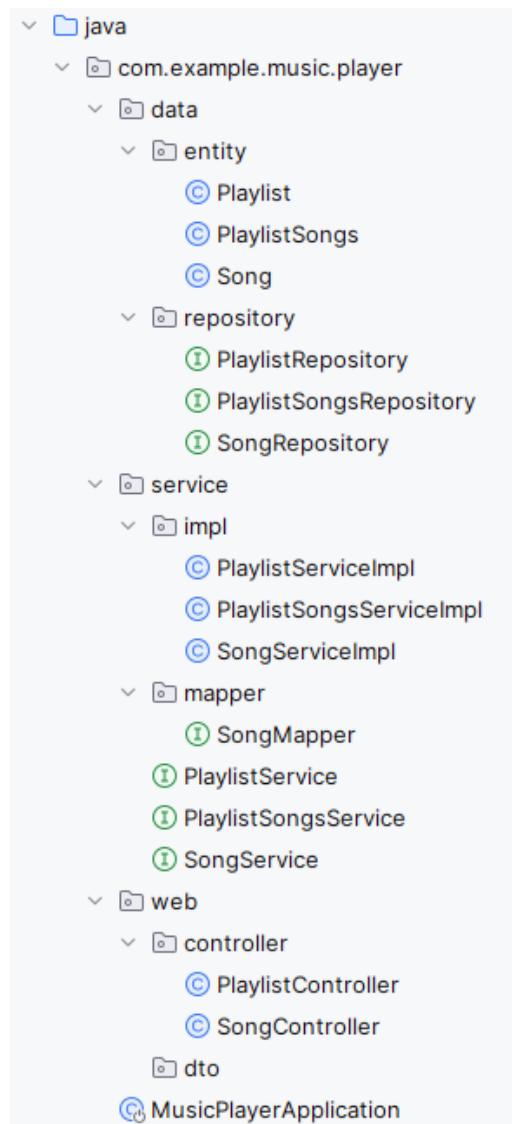


Рис. 1 — Структура проекту сервера

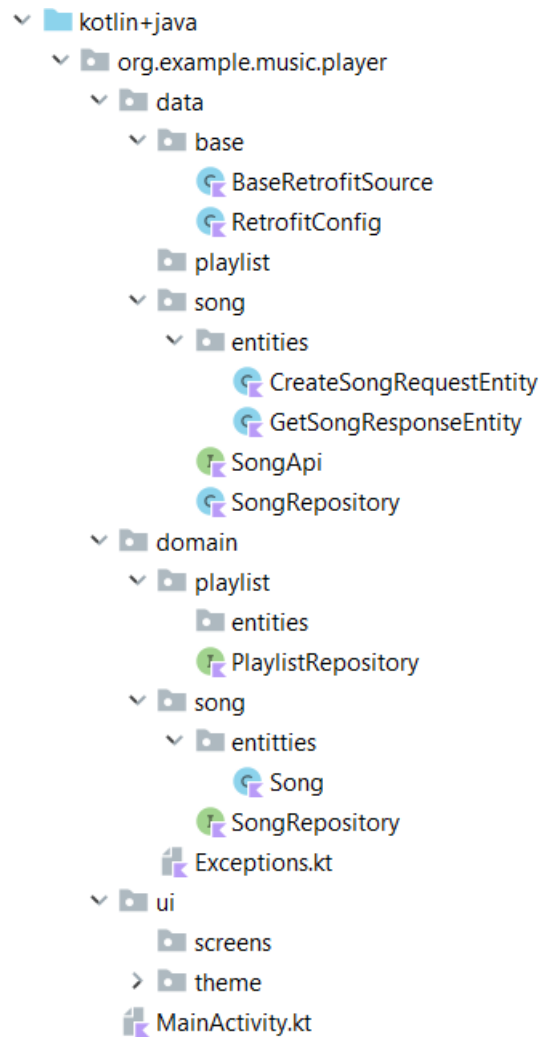


Рис. 2 — Структура проекту клієнта

У наведених структурах проектів ми реалізували серверну (на Java) та клієнтську (на Kotlin) частини для музичного плеєра, які працюють у розподіленому оточенні. Ось деталізований опис кожної частини:

Серверна частина (Java)

Серверна частина побудована за архітектурою багатошарового додатку з чітким розділенням відповідальностей.

Шари серверної архітектури:

1. Entity (Сутності):

- Playlist, PlaylistSongs, Song: Сутності, що відповідають за відображення таблиць бази даних. Вони представляють основні доменні об'єкти — пісні, плейлисти та зв'язки між ними.

2. Repository (Репозиторії):

- `PlaylistRepository`, `PlaylistSongsRepository`, `SongRepository`: Інтерфейси для взаємодії з базою даних. Вони забезпечують CRUD-операції над сутностями, використовуючи ORM.
3. **Service (Сервіси):**
 - Інтерфейси: `PlaylistService`, `PlaylistSongsService`, `SongService` відповідають за бізнес-логіку, абстрагуючи роботу з репозиторіями.
 - Реалізації: `PlaylistServiceImpl`, `PlaylistSongsServiceImpl`, `SongServiceImpl` — реалізують логіку управління плейлистами та піснями.
 4. **Mapper (Мапери):**
 - `SongMapper` та інші мапери виконують перетворення між сутностями та DTO (Data Transfer Object) для передачі даних у клієнт.
 5. **Web (Веб-шар):**
 - Контролери: `PlaylistController`, `SongController` відповідають за обробку HTTP-запитів, наприклад, створення, оновлення та отримання плейлистів чи пісень.
 6. **DTO (Об'єкти передачі даних):**
 - DTO використовуються для передачі даних між клієнтом і сервером. Вони спрощують роботу з даними в API.
 7. **Основний клас:**
 - `MusicPlayerApplication`: Запускає сервер та ініціалізує всі залежності.

Функціонал серверу:

- Управління плейлистами та піснями: створення, оновлення, видалення, отримання.
- Обробка REST-запитів для забезпечення роботи в розподіленому оточенні.
- Підтримка бізнес-логіки для взаємодії між користувачем і базою даних.

Клієнтська частина (Kotlin + Java)

Клієнтська частина реалізована для взаємодії з сервером через API та надання користувацького інтерфейсу.

Шари клієнтської архітектури:

1. **Data (Дані):**
 - `BaseRetrofitSource`, `RetrofitConfig`: Базовий клас і конфігурація для роботи з REST API через Retrofit.

- Entities: CreateSongRequestEntity, GetSongResponseEntity, PlaylistRepository і SongRepository реалізують обробку API-викликів для плейлистів і пісень.
2. Domain (Доменний шар):
 - Entities: Об'єкти доменної моделі, що використовуються в бізнес-логіці.
 - Repositories: Відповідають за взаємодію між Data і UI-шарами, забезпечуючи запити до сервера.
 3. UI (Користувачський інтерфейс):
 - Screens: Екрани для взаємодії з плейлистами та піснями.
 - Theme: Налаштування стилів і тем для додатку.
 - MainActivity: Основна точка входу до додатку.
 4. Exceptions.kt: Визначає винятки для обробки помилок, таких як невірний формат даних чи проблеми з мережею.

Функціонал клієнта:

- Інтерфейс для відтворення, перегляду, створення та редагування плейлистів і пісень.
- Відправка REST-запитів до серверу та обробка відповідей.
- Використання архітектури MVVM (Model-View-ViewModel) для чіткого розділення логіки.

2. Реалізувати взаємодію розподілених частин

```

@RestController
@RequestMapping("/songs")
@AllArgsConstructor
public class SongController {
    private final SongService songService;

    @GetMapping
    public ResponseEntity<List<Song>> getSongs() { return ResponseEntity.ok(songService.getAllSongs()); }

    @GetMapping("/{id}")
    public ResponseEntity<Song> getSong(@PathVariable Long id) {
        return ResponseEntity.ok(songService.getSongById(id));
    }

    @PostMapping
    public ResponseEntity<Song> createSong(Song song) {
        return new ResponseEntity<>(songService.createSong(song), HttpStatus.CREATED);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteSong(@PathVariable Long id) {
        songService.deleteSongById(id);
        return ResponseEntity.noContent().build();
    }
}

```

Рис. 3 — Реалізація АПІ пісень на стороні сервера

```

interface SongApi {

    @GET("songs")
    suspend fun getSongs(): List<GetSongResponseEntity>

    @GET("songs/{id}")
    suspend fun getSong(@Path("id") id: Long): GetSongResponseEntity

    @POST("songs")
    suspend fun createSong(
        createSongRequestEntity: CreateSongRequestEntity
    )

    @DELETE("songs/{id}")
    suspend fun deleteSong(@Path("id") id: Long)
}

```

Рис. 4 — Реалізація АПІ пісень на стороні клієнта

У цьому прикладі взаємодія розподілених частин (клієнта та сервера) реалізована через REST API, де сервер надає кінцеві точки (endpoints) для роботи з ресурсами, а клієнт звертається до них через HTTP-запити. Детально розглянемо, як ця взаємодія працює.

Серверна частина: SongController

Серверна частина реалізує REST API через клас SongController, який:

1. Обробляє HTTP-запити (GET, POST, DELETE) до /songs.
2. Використовує сервіс SongService для виконання бізнес-логіки та роботи з базою даних.

Основні функції сервера:

- Отримання списку пісень:
Метод getSongs() обробляє запит GET /songs, викликає метод сервісу для отримання всіх пісень і повертає результат у форматі JSON.
- Отримання однієї пісні:
Метод getSong(@PathVariable Long id) обробляє запит GET /songs/{id}, знаходить пісню за ID і повертає її як відповідь.
- Створення нової пісні:
Метод createSong(Song song) обробляє запит POST /songs, створює нову пісню через сервіс та повертає її з кодом статусу 201 Created.

- Видалення пісні:

Метод `deleteSong(@PathVariable Long id)` обробляє запит `DELETE /songs/{id}`, викликає сервіс для видалення пісні за ID і повертає код статусу 204 No Content.

Таким чином, сервер виступає провайдером ресурсів, приймаючи запити від клієнта, виконуючи необхідні дії з даними та повертаючи результат.

Клієнтська частина: SongApi

Клієнт реалізує логіку доступу до серверу через інтерфейс `SongApi`, який:

1. Використовує бібліотеку `Retrofit` для генерації HTTP-запитів.
2. Має асинхронну модель виконання запитів (`suspend` функції).

Основні функції клієнта:

- Отримання списку пісень:
Метод `getSongs()` надсилає запит `GET /songs` і отримує список пісень у вигляді `List<GetSongResponseEntity>`.
- Отримання однієї пісні:
Метод `getSong(@Path("id") id: Long)` надсилає запит `GET /songs/{id}` і повертає об'єкт `GetSongResponseEntity`.
- Створення нової пісні:
Метод `createSong(CreateSongRequestEntity createSongRequestEntity)` надсилає запит `POST /songs` з даними нової пісні в тілі запиту.
- Видалення пісні:
Метод `deleteSong(@Path("id") id: Long)` надсилає запит `DELETE /songs/{id}` для видалення пісні за ID.

Клієнт отримує відповіді у форматі JSON, які автоматично десеріалізуються в об'єкти `GetSongResponseEntity` або інші потрібні моделі.

Як реалізована взаємодія:

1. Загальна схема:
 - Клієнт (`SongApi`) викликає методи інтерфейсу, які генерують HTTP-запити до відповідних кінцевих точок сервера.
 - Сервер (`SongController`) отримує запит, обробляє його через сервісний шар і повертає результат у форматі JSON.
2. Формат передачі даних:
 - Дані передаються у форматі JSON через HTTP. Наприклад:
 - При `POST /songs` клієнт передає JSON із полями нової пісні.
 - При `GET /songs` сервер повертає JSON зі списком пісень.

3. Асинхронність:

- Клієнтська частина використовує `suspend` функції, що дозволяє виконувати запити асинхронно, не блокуючи основний потік.

4. Протокол HTTP:

- Взаємодія базується на стандартних методах HTTP (GET, POST, DELETE), що спрощує розподілену роботу, роблячи її стандартизованою та сумісною з різними клієнтами.

5. Розподіленість:

- Клієнт і сервер можуть працювати на різних пристроях чи серверах, обмінюючись даними через HTTP. Це забезпечує незалежність їхніх реалізацій і легке масштабування.

Висновки:

У ході лабораторної роботи було реалізовано клієнт-серверну архітектуру для взаємодії між клієнтом і сервером у застосунку. Це дозволило забезпечити чіткий розподіл обов'язків між компонентами, спростити обробку запитів, полегшити додавання нового функціоналу та створити масштабовану й гнучку систему для розробки музичного програвача.

Код проекту:

<https://github.com/vv0loshyn/TRPZ-labs>